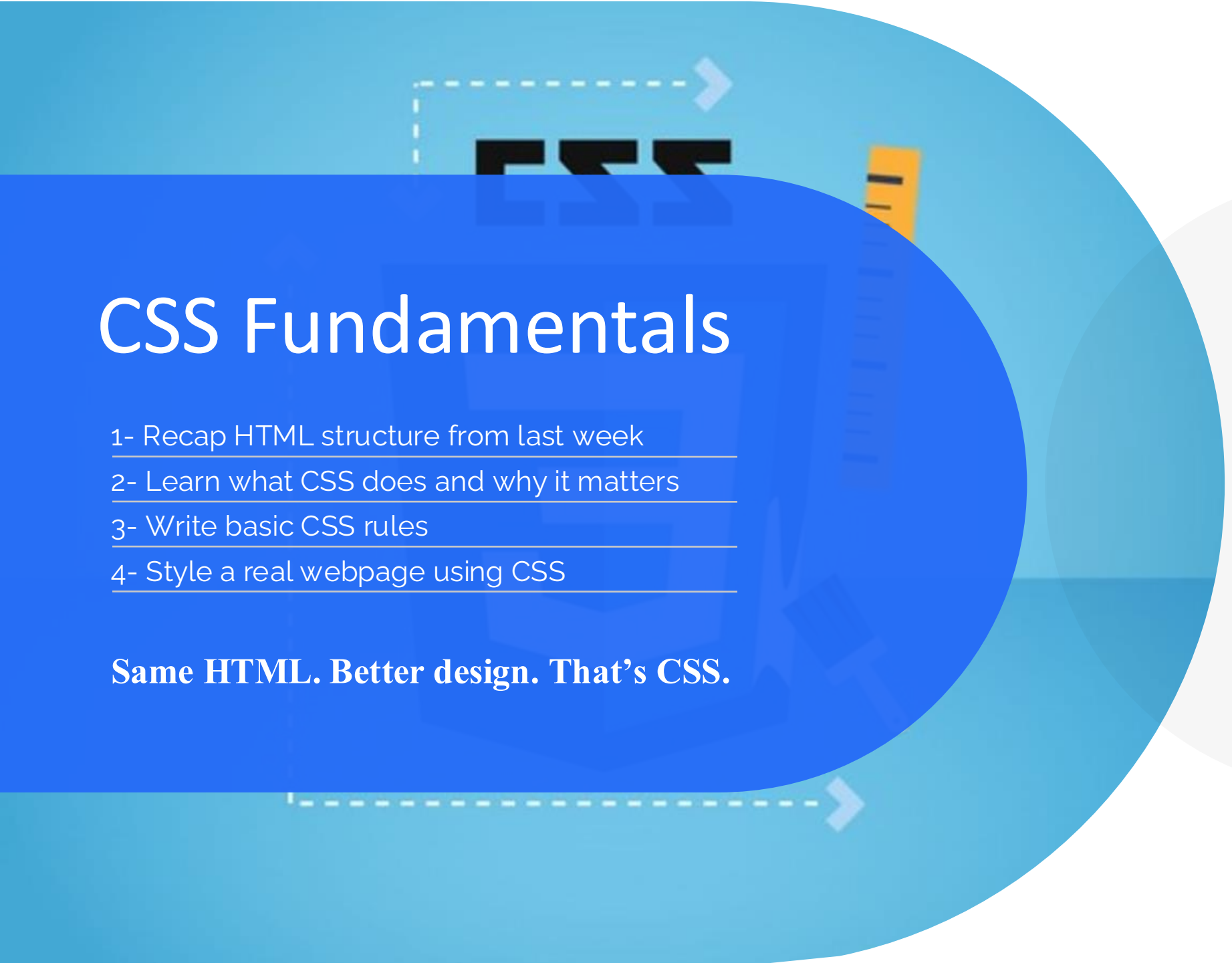


CSS Fundamentals

1- Recap HTML structure from last week

2- Learn what CSS does and why it matters

3- Write basic CSS rules

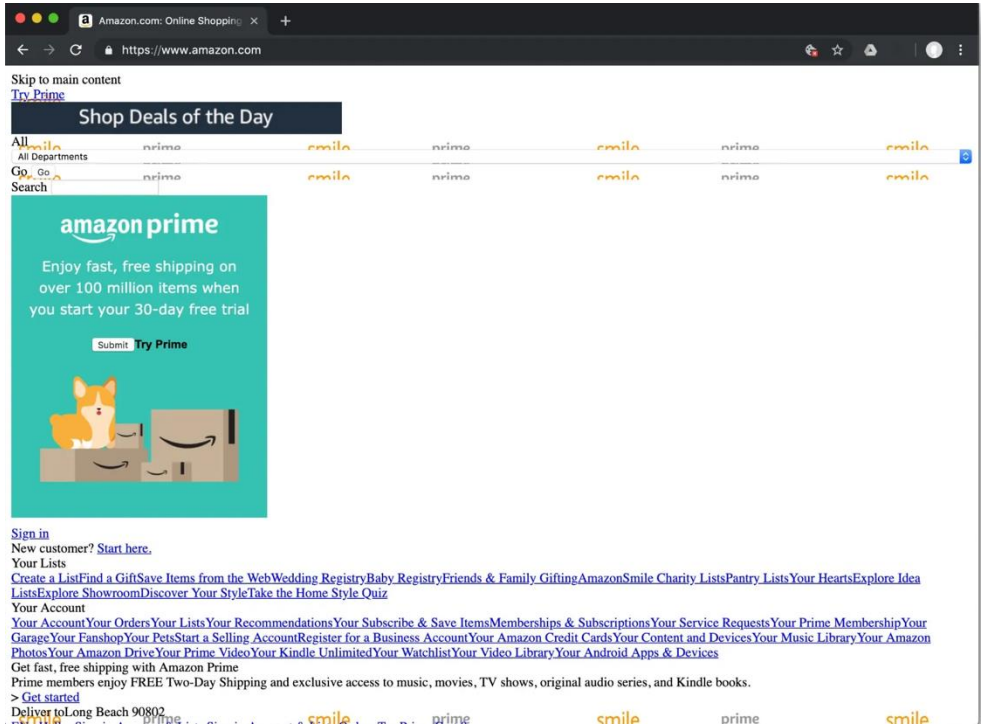
4- Style a real webpage using CSS

Same HTML. Better design. That's CSS.

SCROLL

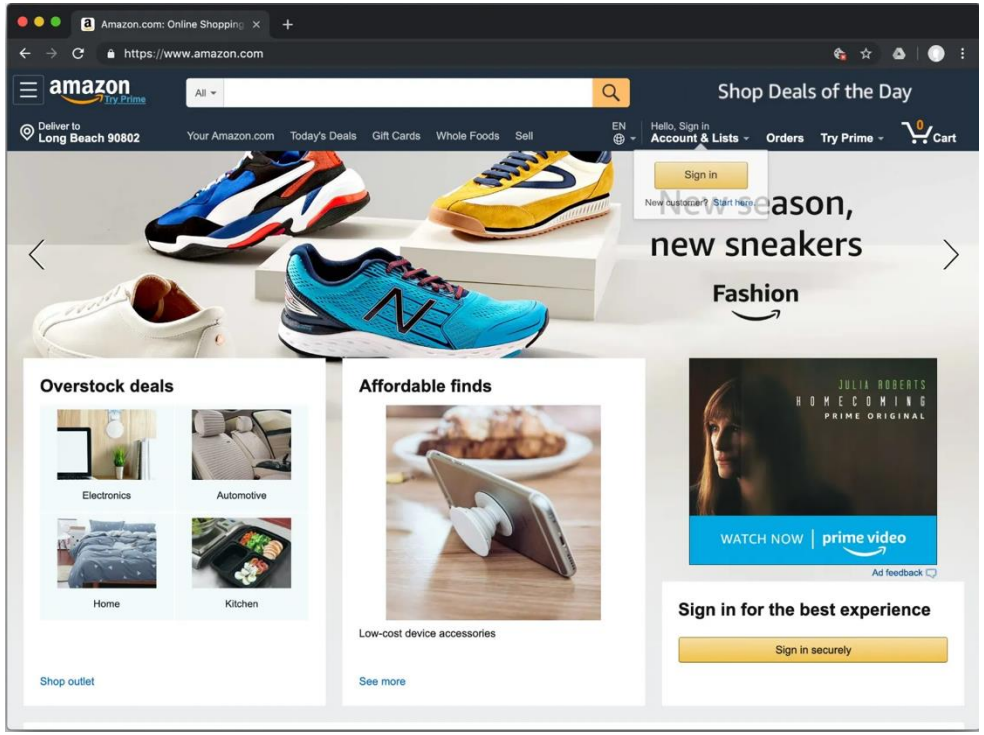
The CSS Transformation

Same HTML. Different CSS.
That's what we're learning today.



BEFORE (HTML only):

- Like a Word doc from 1995
- All structure, no style
- Functional but boring



AFTER (HTML + CSS):

- Professional & beautiful
- Same content, different presentation
- This is the power of CSS!

The House Metaphor Returns



What does CSS stand for?

CSS = Cascading Style Sheets

The House Metaphor

Building a Website = Building a House

HTML = Frame, walls, floors (STRUCTURE) —Last module

CSS = Paint, furniture, decorations (STYLE) — Today's focus!

JavaScript = Electricity, plumbing (FUNCTION) — No included in this class

CSS Syntax Breakdown

```
selector {
    property: value;
    property: value;
}
```

Anatomy of CSS:

- selector → Who you're styling (h1, p, .class)
- property → What you're changing (color, font-size)
- value → Which value you're changing it to (blue, 32px)
- { } → Curly braces contain all styles
- ; → Semicolon ends each line

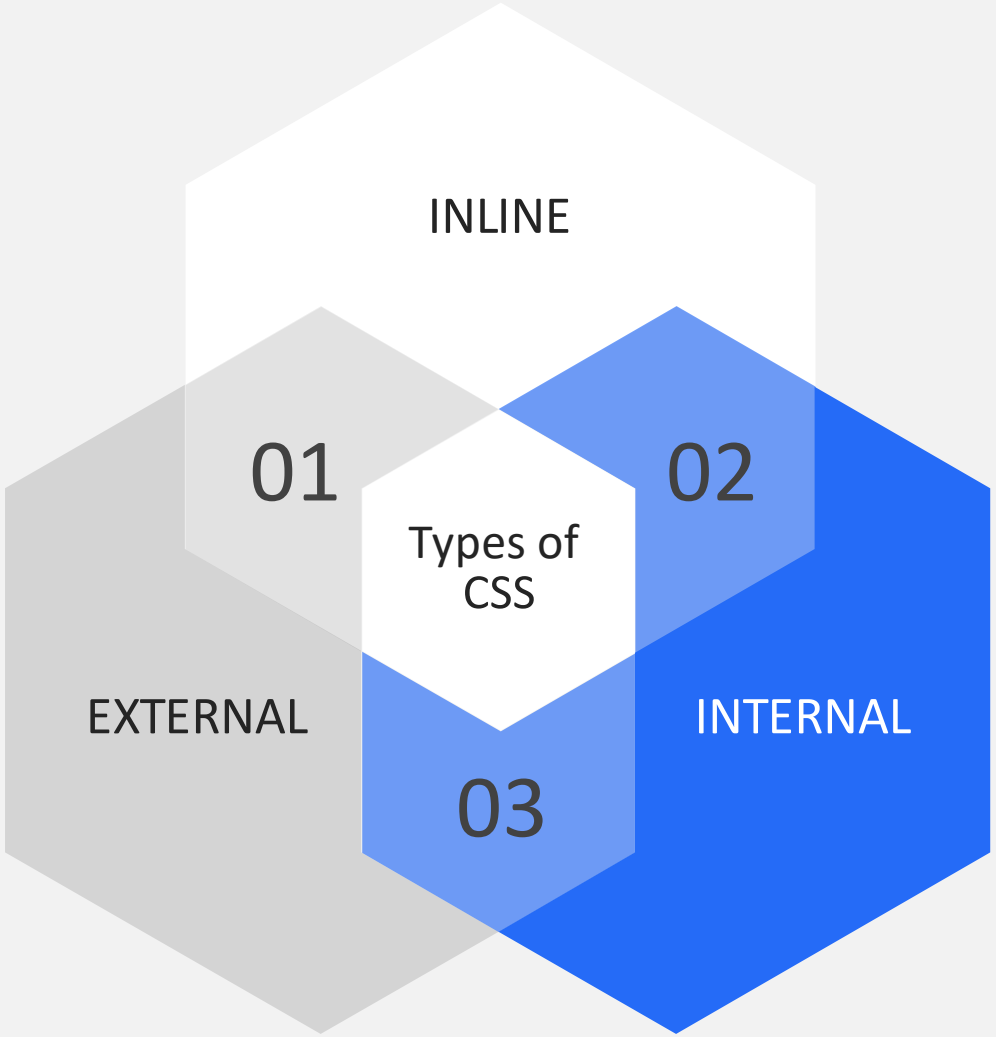
```
h1 {
    color: blue;
    font-size: 32px;
    font-family: 'Courier New', Courier, monospace;
    font-weight: bold;
    text-align: center;
    text-decoration: underline;
    text-transform: uppercase;
}
```

WHO gets WHAT style with WHICH value.
Everything in CSS follows this pattern

Three Ways to Add CSS

How to Add CSS to HTML?

- **Inline (inside HTML tag)**
- **Internal (in <head> section)**
- **External (separate .css file)**



Method 1 — Inline CSS



```
<h1 style="color: blue; font-size: 32px;">About Us</h1>
<h2 style="color: yellow; font-size: 24px;">Our Team</h2>
<h2 style="color: red; font-size: 24px;">Our Mission</h2>
<h2 style="color: gray; font-size: 24px;">Contact</h2>
```

Why it's bad?

- Hard to maintain
- Not reusable
- Mixes HTML and CSS
- Unprofessional

Want to change the color?

You have to edit EVERY. SINGLE. TAG.

Method 2 — Internal CSS



```
<head>
    <style>
        h1 {
            color: blue;
            font-size: 32px;
        }
    </style>
</head>
```

Good for

- Single-page sites
- Learning
- Quick prototypes

Want the same styles on 10 pages?
Copy/paste 10 times.

Method 3 — External CSS



HTML file

```html
<head>
    <link rel="stylesheet" href="style.css">
</head>
```

CSS file

```css
h1 {
    color: blue;
    font-size: 32px;
}
```

Project structure

```
my-website/
  index.html
  about.html
  contact.html
  style.css  ← ONE file styles ALL pages!
```

Benefits:

- One CSS file = styles for entire website
- Change color once = updates everywhere
- Clean, organized, professional
- This is what you'll use in your projects

This is what we'll use. Separates concerns, reusable, maintainable.

CSS Selectors Deep Dive

Basic Selectors:

- Universal selector (*)
- Element selectors (p, h1, div)
- Class selectors (.highlight, .intro)
- ID selectors (#hero, #header)

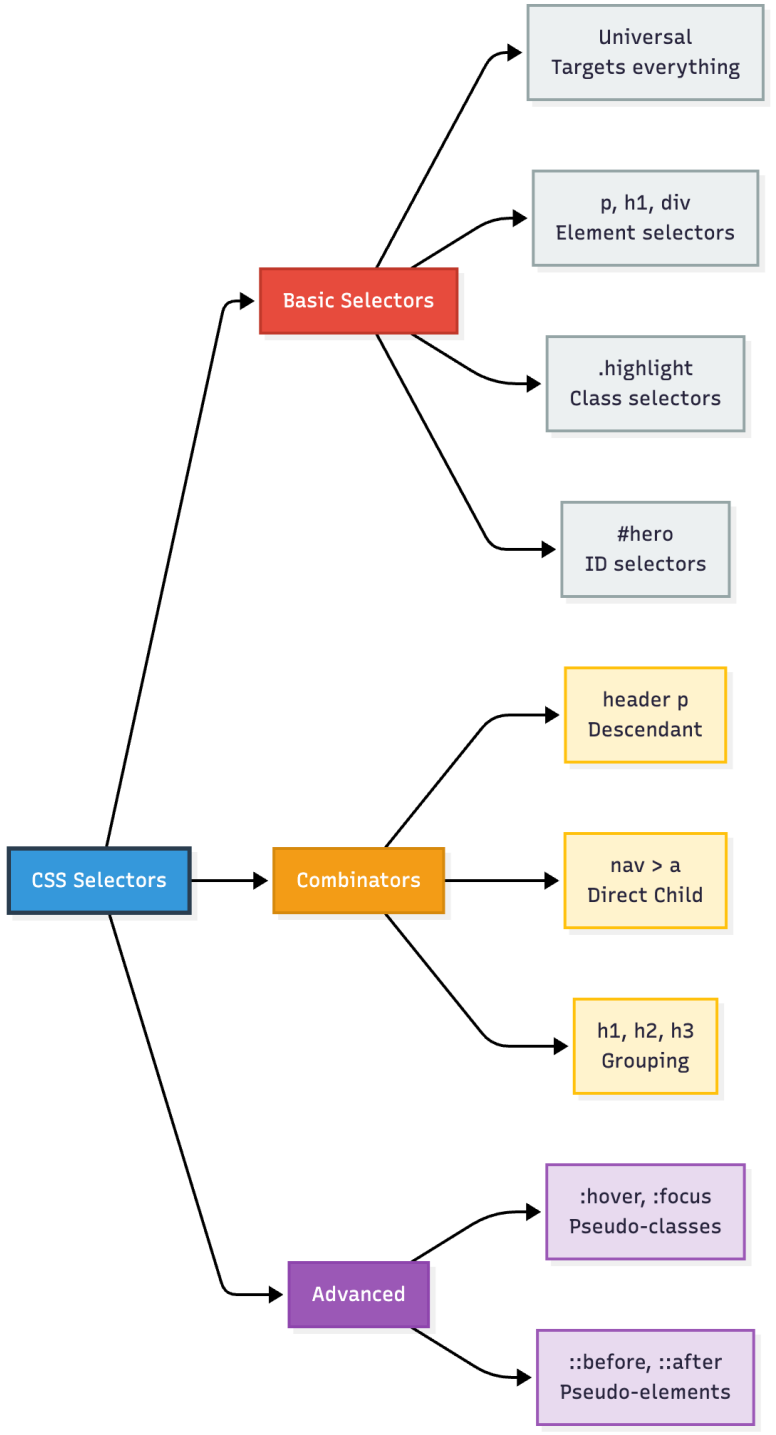
Combinators:

- Descendant selectors (header p)
- Child selectors (nav > a)
- Grouping selectors (h1, h2, h3)

Advanced:

- Pseudo-classes (:hover, :focus)
- Pseudo-elements (::before, ::after)

Selectors are how you target elements to style



Element Selectors

```css
p {
    color: black;
    line-height: 1.6;
}

h1 {
    font-size: 48px;
}

a {
    text-decoration: none;
}
```

Key Points:

- Targets every element of that type on the page
- No special characters needed (no . or # or :)
- Most basic and simple selector
- Use for global styles that apply everywhere

Simplest selector. Affects every element of that type

Class Selectors

HTML

```html
<p class="highlight">Special text</p>
<p>Regular text</p>
<p class="highlight">Another special text</p>
```

CSS

```css
.highlight {
    background-color: yellow;
    font-weight: bold;
}
```

Classes are reusable. Use them for repeating styles.

Key Points:

- Use a dot (.) before the class name in CSS
- Reusable - apply to multiple elements
- Multiple elements can share the same class
- One element can have multiple classes: class="highlight intro"
- Most commonly used selector in professional CSS

ID Selectors

HTML

```html
<div id="hero">Hero content here</div>
```

CSS

```css
#hero {
    font-size: 48px;
    background: linear-gradient(to right, blue, purple);
}
```

Rule: Each ID can only appear once on a page!

Key Points:

- Use a hashtag (#) before the ID name in CSS
- IDs must be UNIQUE - only ONE per page
- More specific than classes (higher specificity = 100 points)
- Best used for JavaScript targeting, not styling
- Use classes for styling, IDs for functionality

Descendant Selectors

```css
header p {
    font-size: 20px;
    color: white;
}

nav a {
    text-decoration: none;
}

main section h2 {
    color: #3498db;
}
```

Key Points:

- Space between selectors = "inside of"
- Targets elements at ANY level deep (children, grandchildren, etc.)
- More specific than element selectors alone
- Reduces the need for extra classes
- Great for styling nested elements

Space between selectors means "inside of."
Very powerful for targeted styling.

Universal Selector

```css
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}

* {
    font-family: Arial, sans-serif;
}
```

Key Points:

- The asterisk (*) means "every single element"
- Applies styles to all elements at once
- Most common use: CSS reset (removes browser defaults)
- Can affect performance on very large sites
- Use strategically, not for everything

The asterisk selector is powerful - it targets every element

Grouping Selector

```css
h1, h2, h3 {
    font-family: 'Montserrat', sans-serif;
    color: #2c3e50;
}

.btn-primary, .btn-secondary, .btn-danger {
    padding: 10px 20px;
    border-radius: 5px;
}

header, footer {
    background-color: #34495e;
    text-align: center;
}
```

Key Points:

- Separate selectors with commas (,)
- All selectors get the same styles
- Saves you from writing duplicate code
- Keeps CSS DRY (Don't Repeat Yourself)
- Can mix element, class, and ID selectors

Instead of writing the same styles three times, group selectors with commas.

Child Combinator (>)

```css
/* Descendant (all levels) */
nav a {
    color: white;  /* ALL links inside nav */
}

/* Child (direct children only) */
nav > a {
    color: white;  /* ONLY direct child links */
}
```

Key Points:

- The > symbol means "direct child only"
- More specific than descendant selectors
- Skips grandchildren and deeper levels
- Gives you precise control over nested structures
- Useful when you have complex HTML nesting

```html
<nav>
    <a href="#">Home</a>  <!-- Styled by BOTH -->
    <div>
        <a href="#">About</a>  <!-- Only styled by nav a -->
    </div>
</nav>
```

The greater-than symbol > means "direct child only"

Pseudo-Classes

Key Points:

- Single colon : indicates a pseudo-class
- Targets special states or positions
- No extra HTML needed - pure CSS magic!
- Makes sites interactive and dynamic
- Different from regular selectors - they react to user actions

Pseudo-classes target special states. The colon : indicates a pseudo-class.

```css
/* Interactive states */
a:hover {
    color: #3498db;  /* Mouse over */
}

a:active {
    color: #e74c3c;  /* Being clicked */
}

input:focus {
    border-color: blue;  /* When selected */
}

/* Position-based */
li:first-child {
    font-weight: bold;  /* First item */
}

li:last-child {
    border-bottom: none;  /* Last item */
}

li:nth-child(even) {
    background-color: #f0f0f0;  /* Zebra stripes */
}
```

Common Pseudo-Classes

Key Points:

- Link order matters! Use LVHA (LoVe HAte): Link → Visited → Hover → Active
- Form pseudo-classes improve user experience
- Position pseudo-classes eliminate the need for extra classes
- Can combine with other selectors: .menu li:hover

Pro Tip: Wrong order = styles won't work correctly!

```css
/* Link States (in this order!) */
a:link { color: blue; }      /* Unvisited */
a:visited { color: purple; } /* Visited */
a:hover { color: red; }      /* Mouse over */
a:active { color: orange; }  /* Being clicked */

/* Form States */
input:focus {
    border-color: #3498db;
    outline: 2px solid #3498db;
}

input:disabled {
    background-color: #e0e0e0;
}

input:checked {
    background-color: green;
}

/* Position States */
p:first-child { margin-top: 0; }
p:last-child { margin-bottom: 0; }
li:nth-child(odd) { background: #f5f5f5; }
```

Pseudo-Elements

Key Points:

- Double colon :: for pseudo-elements (vs single : for pseudo-classes)
- content property is REQUIRED for ::before and ::after
- Adds decorative content without extra HTML
- Great for icons, quotes, and design flourishes
- The content is not selectable by users

Pseudo-elements let you add content with CSS alone - no HTML needed!

```css
.quote::before {
    content: '"';
    font-size: 2em;
    color: #3498db;
}

.quote::after {
    content: '"';
    font-size: 2em;
    color: #3498db;
}

/* Add decorative elements */
h2::before {
    content: "→ ";
    color: #e74c3c;
    margin-right: 10px;
}

/* Style first letter */
p::first-letter {
    font-size: 3em;
    font-weight: bold;
    float: left;
}

/* Style first line */
p::first-line {
    font-weight: bold;
    color: #2c3e50;
}
```

Selector Specificity

Key Points:

- Higher specificity = wins the style battle
- If equal specificity, last one in CSS wins
- Avoid inline styles - they override everything!
- This is why we prefer classes over IDs for styling
- You can calculate: count IDs, classes, and elements

Pro Tip: Keep specificity LOW for easier maintenance!

```
/* Inline style - 1000 points */
/* <p style="color: red;">  DON'T USE! */

/* ID - 100 points */
#hero { color: blue; }

/* Class, pseudo-class, attribute - 10 points */
.highlight { color: yellow; }
:hover { color: green; }
[type="text"] { color: orange; }

/* Element, pseudo-element - 1 point */
p { color: black; }
::before { color: gray; }

/* Universal - 0 points */
* { color: pink; }
```

Attribute Selectors

Key Points:

- Use square brackets [ ] for attribute selectors
- Target elements based on HTML attributes
- Can match exact values, prefixes, suffixes, or partial text
- Great for styling different input types automatically
- Useful for external vs internal links

Common Use Cases:

- Style different input types (text, email, password)
- Add icons to external links or file downloads
- Target links to specific websites

They're great when you want to automatically style elements based on their attributes without adding classes.

```css
/* Exact match */
input[type="text"] {
    border: 1px solid gray;
    padding: 5px;
}

input[type="email"] {
    border: 1px solid blue;
}

/* Starts with ^ */
a[href^="https"] {
    color: green;  /* Secure links */
}

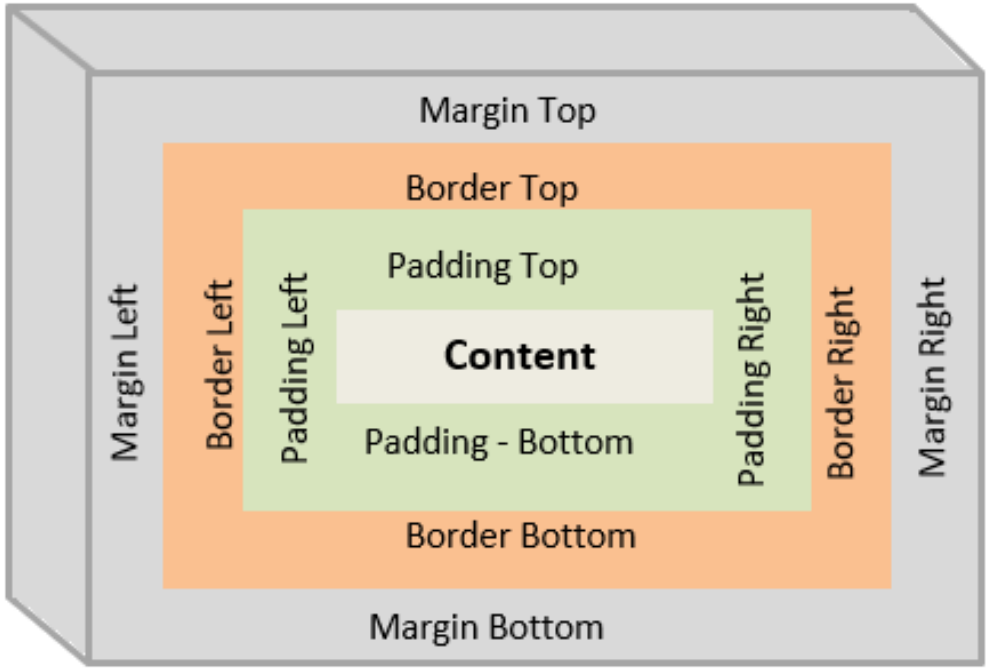
a[href^="mailto"] {
    color: #e74c3c;  /* Email links */
}

/* Ends with $ */
a[href$=".pdf"] {
    color: red;  /* PDF links */
    font-weight: bold;
}

/* Contains * */
a[href*="youtube"] {
    color: #ff0000;  /* YouTube links */
}

a[href*="github"] {
    color: #333;  /* GitHub links */
}
```

The Box Model



This is THE most important concept in CSS. If you understand the box model, everything else clicks into place

Every HTML element is a box with 4 parts:

- **Content** - Your text or image
- **Padding** - Space inside (around content)
- **Border** - Line around the padding
- **Margin** - Space outside (between elements)

Why This Matters:

- Controls ALL spacing and sizing on your page
- Determines how elements fit together
- Foundation for layouts
- Once you understand this, CSS makes sense!

Live Box Model Demo

HTML

```html
<div class="box-demo">
    This a box model!
</div>
```

CSS

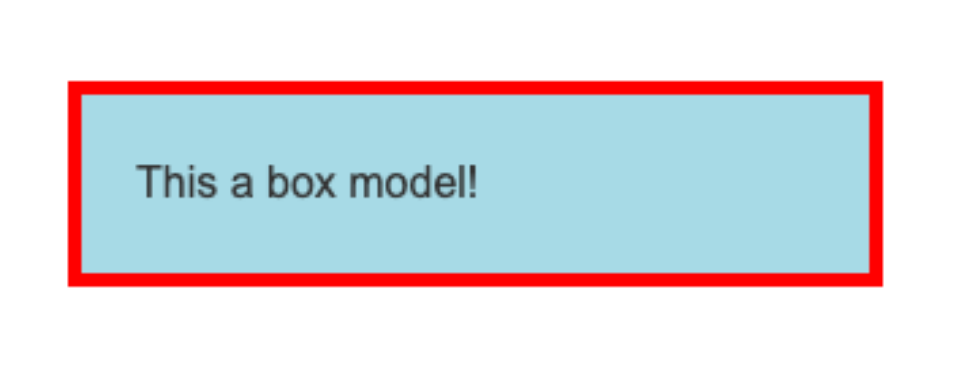
```css
.box-demo {
    width: 300px;
    padding: 20px;
    border: 5px solid red;
    margin: 30px;
    background-color: lightblue;
}
```

What Each Property Does:

- `width: 300px` - content area is 300px wide
- `padding: 20px` - 20px space INSIDE around content
- `border: 5px solid red` - red line around padding
- `margin: 30px` - 30px space OUTSIDE around element
- `background-color` - fills content + padding area

Add this to your CSS file and see the magic!

Box Model Breakdown



Color-Coded Areas:

- Blue area = Content + Padding (text sits here)
- Red line = Border (5px solid red)
- Space around = Margin (30px on all sides))

Important Math:

- Content width: 300px
  - Padding: 20px left + 20px right = 40px
  - Border: 5px left + 5px right = 10px
- Total element width = 350px!

Key Insight: The element takes up MORE space than just the width!

Open DevTools (F12) to see it visually! Right-click the box → Inspect → See the box model diagram

Padding Syntax

Memory Trick:

- TRouBLe = Top, Right, Bottom, Left
- Or think of a clock starting at 12 and going clockwise!

Most Common:
padding: 20px; or padding: 20px 40px;

```
/* All sides the same */
padding: 20px;
/* 20px on all 4 sides */

/* Vertical | Horizontal */
padding: 10px 20px;
/* 10px top/bottom, 20px left/right */

/* Top | Right | Bottom | Left (clockwise!) */
padding: 10px 15px 20px 25px;
/* Goes around like a clock: ⬆️ ➡️ ⬇️ ⬅️ */

/* Individual sides */
padding-top: 10px;
padding-right: 15px;
padding-bottom: 20px;
padding-left: 25px;
/* Specific control for each side */
```

Margin Syntax

Memory Trick:

- `auto` value works with margin (centers elements!)
- Margins can be negative (pull elements closer)
- Margins collapse vertically (more on this later)

Most Common:
`margin: 20px auto;` for centered content with top/bottom spacing

```css
/* All sides the same */
margin: 20px;
/* 20px space around all sides */

/* Vertical | Horizontal */
margin: 10px 20px;
/* 10px top/bottom, 20px left/right */

/* Top | Right | Bottom | Left (clockwise!) */
margin: 10px 15px 20px 25px;
/* ⬆️ ➡️ ⬇️ ⬅️ */

/* Individual sides */
margin-top: 10px;
margin-bottom: 20px;
margin-left: 15px;
margin-right: 15px;

/* Magic centering! */
margin: 0 auto;
/* 0 top/bottom, auto left/right = CENTERED! */

/* Remove margin */
margin: 0;
/* No spacing (useful for resets) */
```

Border Syntax

Border (Line Around Element)

Shorthand: width | style | color

Common Border Styles:

- `solid` - solid line (most common)
- `dashed` - dashed line - - - -
- `dotted` - dotted line · · · ·
- `double` - double line ═══
- `none` - no border

Border adds to element width (unless using box-sizing: border-box); Borders are great for debugging layouts

```css
/* All sides, all-in-one */
border: 2px solid black;
border: 1px dashed red;
border: 3px dotted blue;
border: 5px double green;

/* Individual sides */
border-top: 1px solid black;
border-bottom: 2px solid red;
border-left: 3px dashed blue;
border-right: 1px dotted green;

/* Individual properties */
border-width: 2px;
border-style: solid;
border-color: black;

/* Rounded corners */
border-radius: 10px;  /* Slightly rounded */
border-radius: 20px;  /* More rounded */
border-radius: 50%;   /* Perfect circle! */
border-radius: 10px 20px 30px 40px;

/* Remove border */
border: none;
```

Width Syntax Height

Controlling Element Size

Key Concepts:

- Fixed: width: 300px - always 300px (not responsive)
- Percentage: width: 50% - half of parent width (responsive!)
- max-width: Prevents overflow on small screens
- min-width: Ensures minimum size
- auto: Browser calculates based on content

Pro Tips:

- Always use max-width: 100% on images for responsive design
- Use max-width instead of width for flexible containers
- Height usually best left as auto (let content decide)
- Avoid fixed heights unless it’s necessary

Max-width prevents overflow on small screens. This is key for responsive design. Fixed widths (300px) don't adapt to screen size, percentages do

```css
.box {
    /* Fixed sizes */
    width: 300px;
    height: 200px;

    /* Percentage (relative to parent) */
    width: 50%;
    height: 100%;

    /* Responsive sizes */
    max-width: 100%;   /* Can't be wider than parent */
    min-width: 200px;  /* Can't be narrower than 200px *
    max-height: 500px; /* Can't be taller than 500px */
    min-height: 400px; /* Can't be shorter than 400px */

    /* Auto (browser decides) */
    width: auto;       /* Fits content */
    height: auto;      /* Fits content */
}

/* Responsive images - SUPER IMPORTANT! */
img {
    max-width: 100%;
    height: auto;     /* Maintains aspect ratio */
}

/* Full-width containers */
.container {
    width: 100%;
    max-width: 1200px; /* Limits max width */
    margin: 0 auto;    /* Centers it */
}
```

Colors in CSS

Making your designs come alive!

Four Ways to Specify Colors:

- Color names (red, blue, lightgray)
- Hex codes (#FF0000, #3498db)
- RGB / RGBA (rgb(255, 0, 0), rgba(255, 0, 0, 0.5))
- HSL / HSLA (hsl(200, 70%, 50%), hsla(200, 70%, 50%, 0.8))

Why so many methods?

- Different methods for different needs
- Some easier for humans, some for computers
- Transparency options (RGBA, HSLA)
- Personal preference



Crimson Cyan DarkBlue DarkCyan DarkGoldenRod DarkGray DarkGrey DarkGreen DarkKhaki
DarkMagenta DarkOliveGreen DarkOrange DarkOrchid DarkRed DarkSalmon DarkSeaGreen
DarkSlateBlue DarkSlateGray DarkSlateGrey DarkTurquoise DarkViolet DeepPink DeepSkyBlue
DimGray DimGrey DodgerBlue FireBrick FloralWhite ForestGreen Fuchsia Gainsboro
GhostWhite Gold GoldenRod Gray Grey Green GreenYellow HoneyDew HotPink IndianRed
Indigo Ivory Khaki Lavender LavenderBlush LawnGreen LemonChiffon LightBlue LightCoral
LightCyan LightGoldenRodYellow LightGray LightGrey LightGreen LightPink LightSalmon
LightSeaGreen LightSkyBlue LightSlateGray LightSlateGrey LightSteelBlue LightYellow Lime
LimeGreen Linen Magenta Maroon MediumAquaMarine MediumBlue MediumOrchid
MediumPurple MediumSeaGreen MediumSlateBlue MediumSpringGreen MediumTurquoise
MediumVioletRed MidnightBlue MintCream MistyRose Moccasin NavajoWhite Navy OldLace
Olive OliveDrab Orange OrangeRed Orchid PaleGoldenRod PaleGreen PaleTurquoise
PaleVioletRed PapayaWhip PeachPuff Peru Pink Plum PowderBlue Purple RebeccaPurple Red
RosyBrown RoyalBlue SaddleBrown Salmon SandyBrown SeaGreen SeaShell Sienna Silver

Color Names

Color Names (Easy but Limited)

Pros:

- Easy to remember and type
- Great for quick tests and prototypes
- Readable code
- Perfect for learning

Cons:

- Only 140 options (limited palette)
- Can't adjust exact shades
- No transparency option
- Not professional for final designs

```
color: red;
color: blue;
background-color: lightblue;
border-color: navy;
color: darkgray;
color: coral;
```

How Many Colors?

- 140 named colors available
- Examples: red, blue, green, yellow, orange, purple
- Shades: lightblue, darkblue, navy, skyblue
- Grays: black, white, gray, darkgray, lightgray
- Fun names: coral, salmon, tomato, chocolate, gold

Hex Codes

Hex Codes (Most Common!) #

Pros:

- Industry standard (everyone uses these)
- Millions of color options
- Compact format (6 characters)
- Copy/paste from design tools

Cons:

- Not intuitive to read or create manually
- Hard to guess what color #8e44ad is
- No transparency option (use RGBA for that)

```
color: #FF0000; /* Red */
color: #00FF00; /* Green */
color: #0000FF; /* Blue */
color: #3498db; /* Nice blue */
color: #2c3e50; /* Dark blue-gray */
color: #e74c3c; /* Red-orange */

/* Shorthand (when pairs match) */
color: #fff; /* White (#FFFFFF) */
color: #000; /* Black (#000000) */
color: #f00; /* Red (#FF0000) */
```

#RRGGBB

```
|||||
|||||└─ Blue (00-FF)
||||└── Blue
|||└─── Green (00-FF)
||└──── Green
|└───── Red (00-FF)
└────── Red
```

= "This is a hex color"
00 = No color (0)
FF = Maximum color (255)

Pro Tip: Never try to write hex codes from memory - always use a color picker!

RGB and RGBA

RGB (Red, Green, Blue) / How computers think about color - mixing light!

Pros:

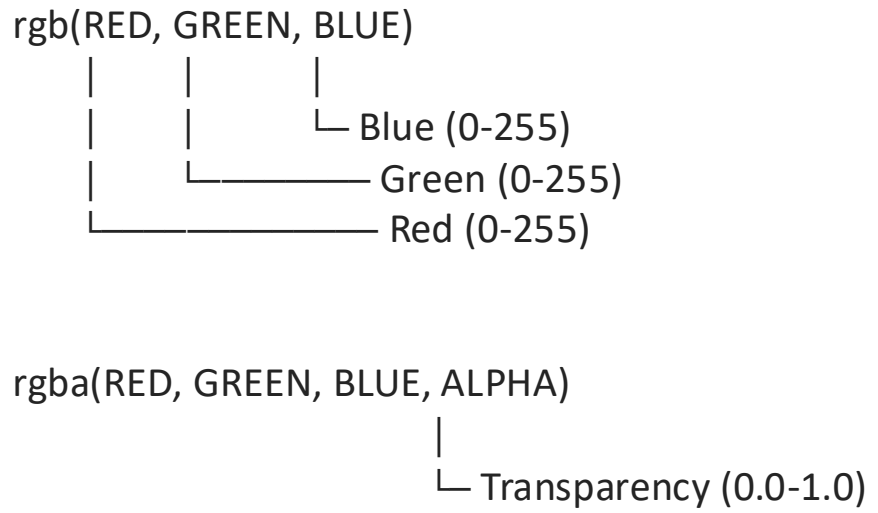
- Intuitive number range (0-255)
- Transparency with RGBA!
- Easy to adjust individual color channels
- Great for overlays and layers

Cons:

- Hard to predict colors from numbers
- More typing than hex codes

```
/* RGB - Three values (0-255) */
color: rgb(255, 0, 0);     /* Red */
color: rgb(0, 255, 0);     /* Green */
color: rgb(0, 0, 255);     /* Blue */
color: rgb(52, 152, 219); /* Nice blue */
color: rgb(231, 76, 60);   /* Red-orange */

/* RGBA = RGB + Alpha (transparency) */
color: rgba(255, 0, 0, 0.5); /* 50% transparent red */
color: rgba(0, 0, 0, 0.8);   /* 80% opaque black */
background: rgba(52, 152, 219, 0.3); /* Subtle blue overlay */
```



HSL and HSLA

HSL (Hue, Saturation, Lightness) / How humans think about color - most intuitive!

Pros:

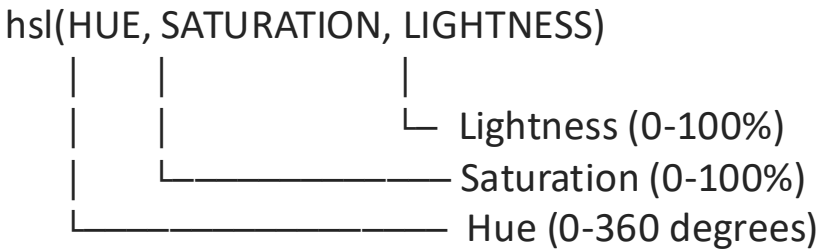
- Most intuitive for humans to understand
- Easy to create color variations (just change one number!)
- Perfect for color schemes and palettes
- Transparency with HSLA
- Modern and powerful

Cons:

- Less common
- Not all designers know it yet

```
/* HSL - Three values */
color: hsl(0, 100%, 50%);   /* Red */
color: hsl(120, 100%, 50%); /* Green */
color: hsl(240, 100%, 50%); /* Blue */
color: hsl(200, 70%, 50%);  /* Nice blue */
color: hsl(10, 80%, 60%);   /* Coral/salmon */

/* HSLA = HSL + Alpha (transparency) */
color: hsla(200, 70%, 50%, 0.8);  /* Blue, 80% opaque */
background: hsla(0, 0%, 0%, 0.5); /* 50% transparent gray */
```



Color Tools

AI-Powered Color Tools (NEW!):

- Huemint (huemint.com) - AI creates palettes using machine learning
- ColorMagic (colormagic.app) - Type keywords, AI generates palettes ("sunset beach", "cyberpunk")

Palette Generators:

- Coolors.co - Hit spacebar for instant palettes!
- Adobe Color (color.adobe.com) - Color wheel + harmony rules
- Happy Hues (happyhues.co) - Palettes with real website examples

Accessibility Checkers (IMPORTANT!):

- WebAIM Contrast Checker (webaim.org/resources/contrastchecker)
- Coolors Contrast Checker - Built into Coolors
- Who Can Use (whocanuse.com) - Shows how people with different vision see your colors

Browser Extensions:

- ColorZilla - Eyedropper + color picker
- Eye Dropper - Pick colors from any website

Pro Workflow:

- Start with AI tool (Khroma/ColorMagic) or inspiration site
- Refine in Coolors or Adobe Color
- Check contrast with accessibility checker
- Copy hex codes to your CSS!

Color Best Practices

The 60-30-10 Rule:

- 60% - Primary/dominant color (backgrounds, large areas)
- 30% - Secondary color (sections, cards)
- 10% - Accent color (buttons, links, emphasis)

Real-World Examples:

- Facebook - Blue (#1877f2) + white + gray (simple, recognizable)
- Spotify - Green (#1DB954) + black (bold, modern)
- Stripe - Purple (#635bff) + subtle grays (professional, trustworthy)

DO:

- Limit to 2-3 main colors (+ neutrals like black, white, gray)
- Ensure good contrast - Text must be readable!
  - Normal text: 4.5:1 contrast ratio minimum
  - Large text (18px+): 3:1 contrast ratio minimum
- Use color for emphasis - Highlight important things
- Stay consistent - Same colors for same purposes
- Test in grayscale - Design should work without color
- Consider color blindness - Don't rely on color alone to convey meaning

DON'T:

- Too many colors - Looks chaotic and unprofessional
- Low contrast - Gray text on light gray background = unreadable
- Pure black (#000) on pure white (#fff) - Too harsh, use #333 instead
- Neon/bright colors everywhere - Hard on the eyes
- Random color choices - Pick colors intentionally
- Ignore accessibility - 1 in 12 men are colorblind!

Background Properties

Background Styling Beyond Colors

Key Points:

- cover = fills container, may crop image
- contain = shows full image, may leave gaps
- center = positions image in middle
- Use no-repeat for photos/hero images
- Combine color + image for fallback

Pro Tips:

- Always set background-color as fallback if image doesn't load
- Use cover for full-screen hero sections
- Optimize images for web (compress them!)
- fixed attachment creates cool parallax effects

```css
/* Background color */
background-color: #3498db;

/* Background image */
background-image: url('image.jpg');

/* Position - where image sits */
background-position: center;
background-position: top right;
background-position: 50% 50%;

/* Size - how image scales */
background-size: cover;      /* Fills entire area */
background-size: contain;    /* Fits inside, may show gaps */
background-size: 100% 100%;  /* Stretches to fit */

/* Repeat - tiling behavior */
background-repeat: no-repeat;  /* Show once */
background-repeat: repeat;     /* Tile everywhere */
background-repeat: repeat-x;   /* Horizontal only */

/* Attachment - scroll behavior */
background-attachment: fixed;  /* Parallax effect! */
background-attachment: scroll; /* Moves with content */
```


CSS syntax and how it works?
Linking CSS files?
Colors (hex, RGB, HSL)?
Background properties?
CSS Selectors?
The Box Model?
HAVE ANY
QUESTIONS?